AI Capstone: Milestone 4 Report

Aliyyah Jackhan, Aadil Shaikh, Jonathan Chacko

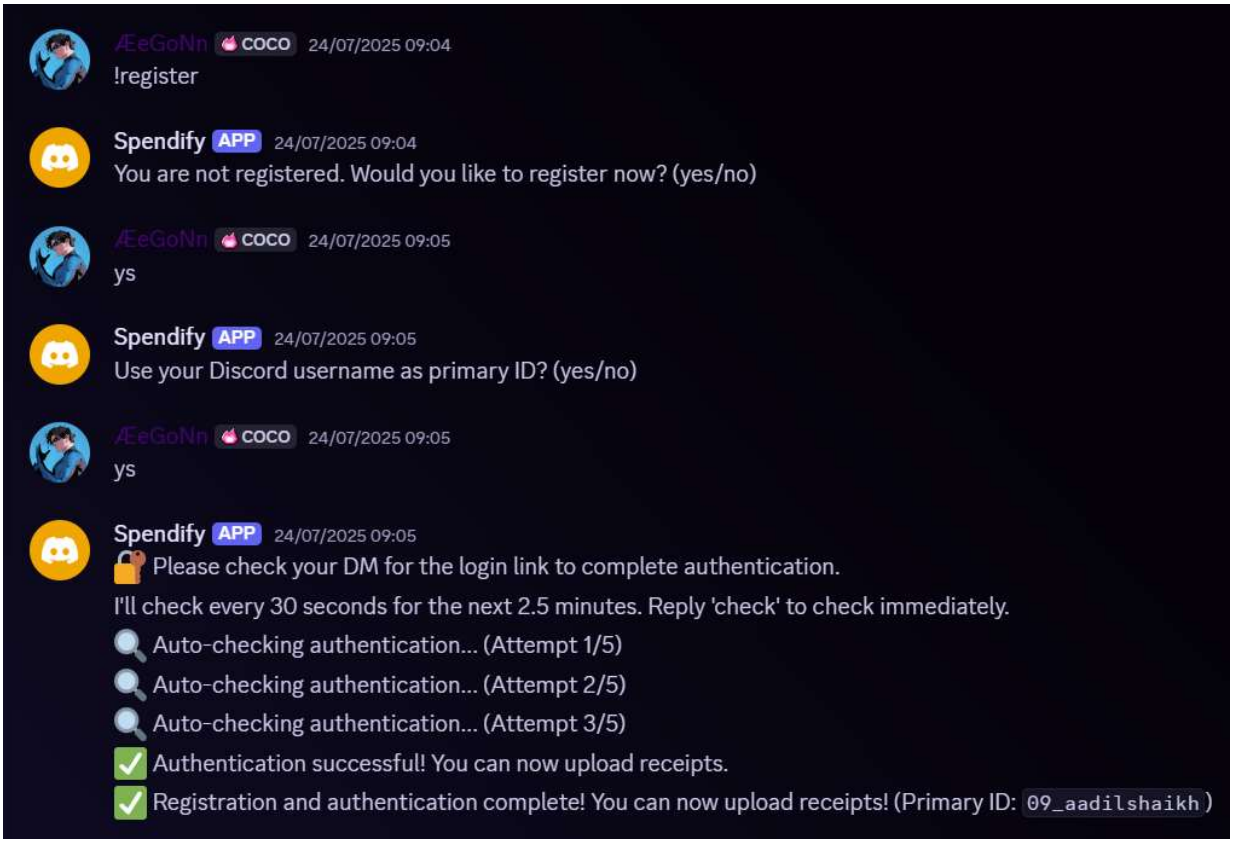
1. End-to-End System Demonstration

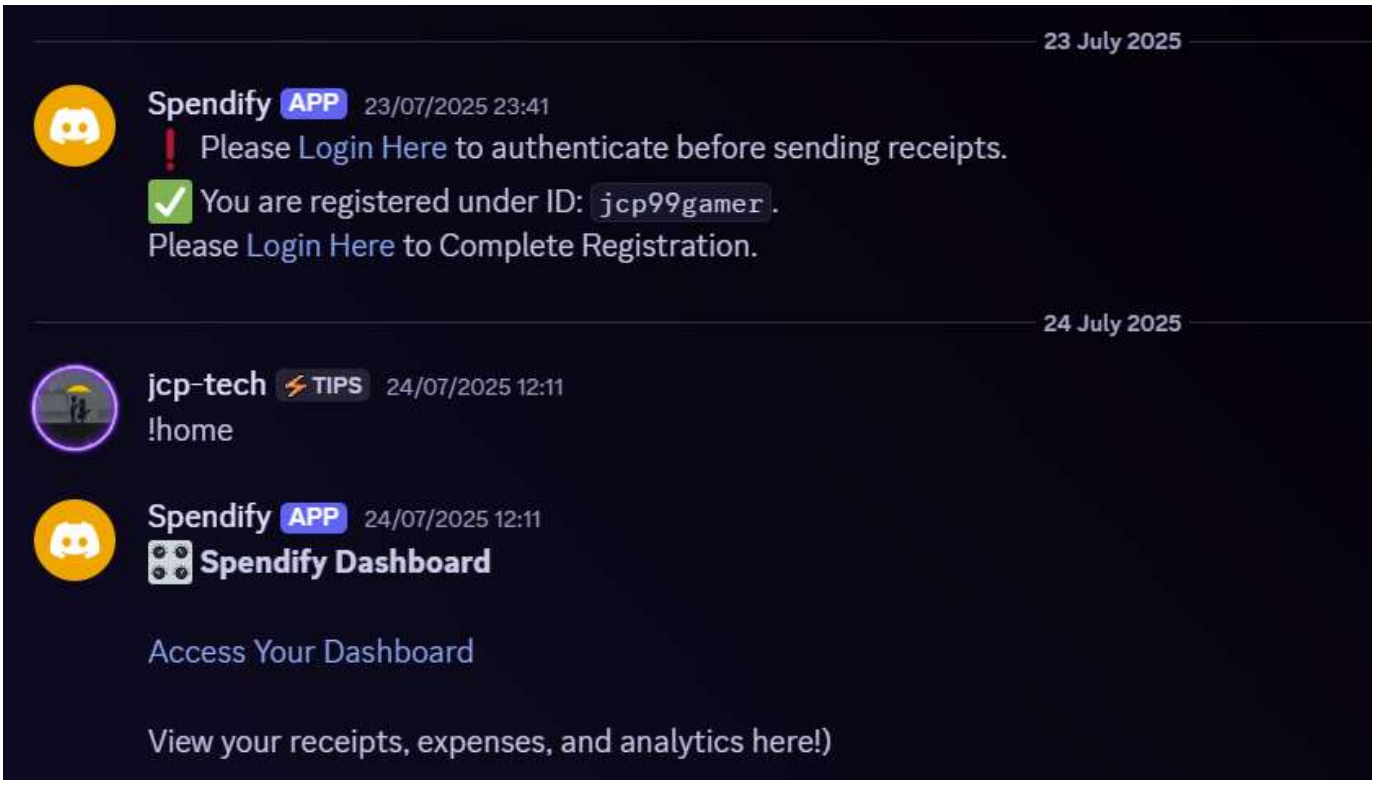
The system's core function is demonstrated through live API calls and an integrated demo. The system's workflow begins with a user uploading receipts in Discord and ends with the user receiving classified results and summaries. The data flows from the initial upload through structured analysis and storage.

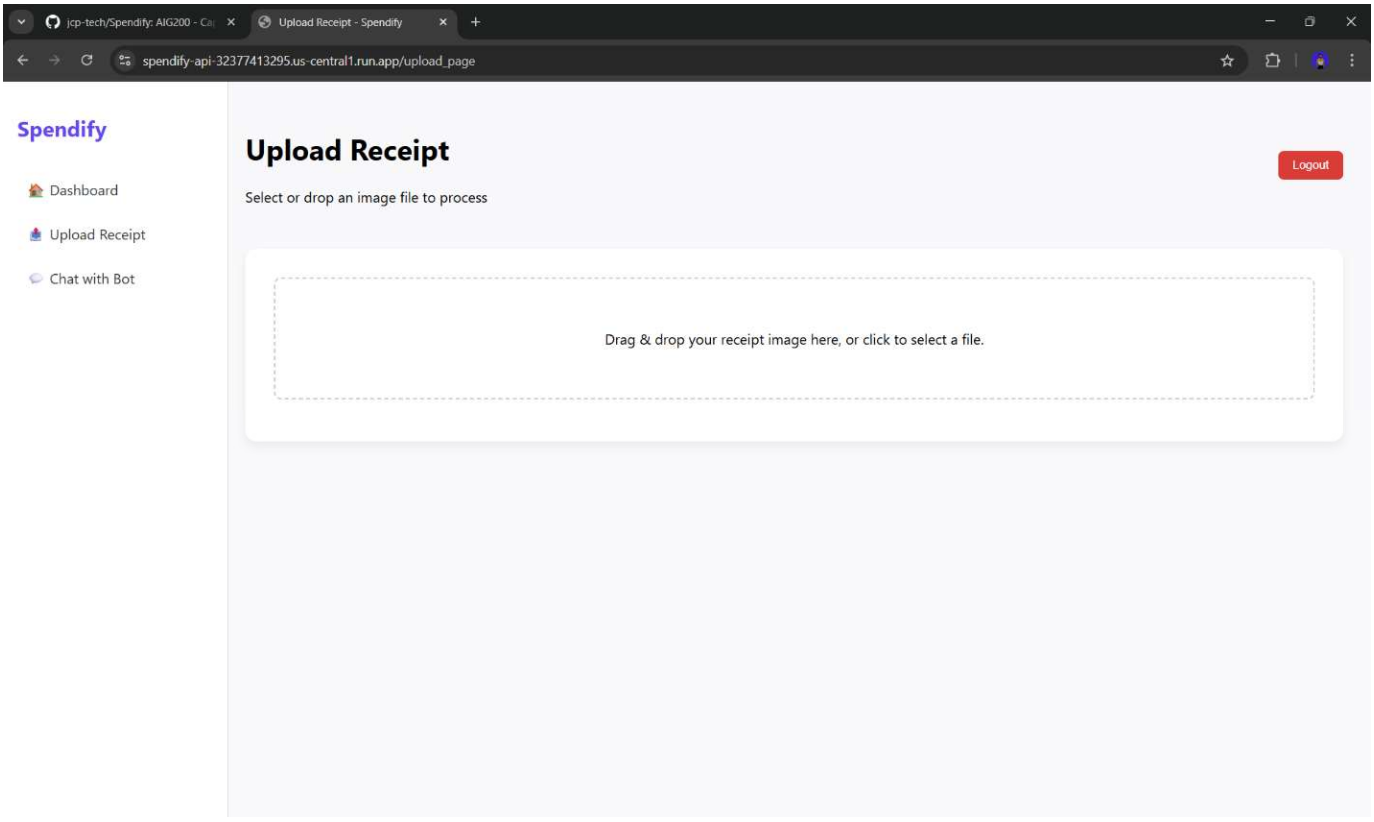
The system's components work together as follows:

- A Discord Bot receives user input.
- The Flask API handles the requests.
- Document AI (OCR) extracts data from the receipts.
- The ADK Classification Pipeline processes the data.
- Firebase DB stores the final results.

Screenshots of the Discord bot and the web-based "Upload Receipt" interface are provided below.




23 July 2025
Spendify APP 23/07/2025 23:41
Please Login Here to authenticate before sending receipts.
You are registered under ID: jcp99gamer.
Please Login Here to Complete Registration.
24 July 2025
jcp-tech TIPS 24/07/2025 12:11
!home
Spendify APP 24/07/2025 12:11
Spendify Dashboard
Access Your Dashboard
View your receipts, expenses, and analytics here!)


Upload Receipt - Spendify
spendify-api-32377413295.us-central1.run.app/upload_page
Spendify
Dashboard
Upload Receipt
Chat with Bot
Upload Receipt
Logout
Select or drop an image file to process
Drag & drop your receipt image here, or click to select a file.

2. Deployment Status

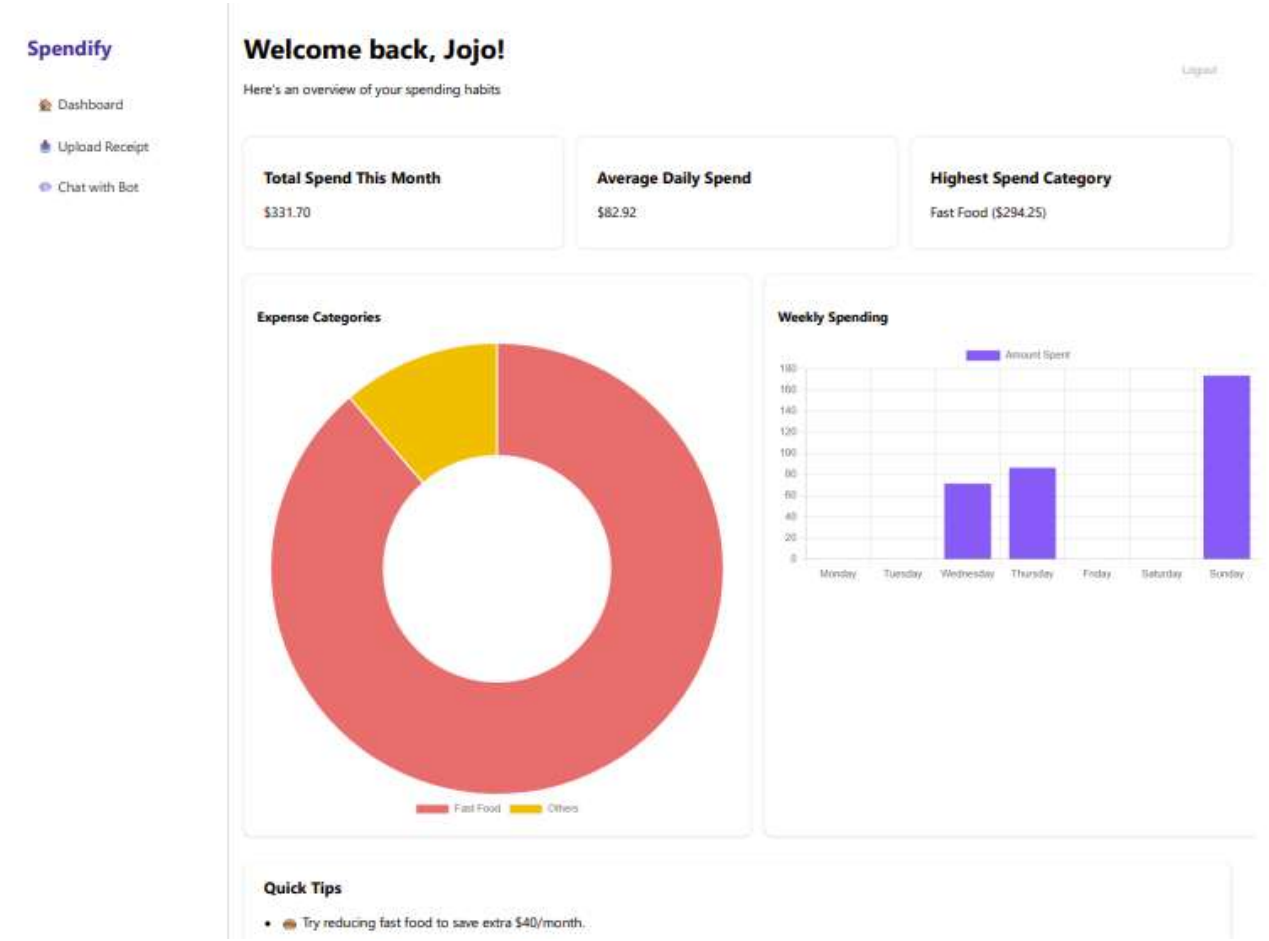
The system has been successfully deployed to Google Cloud, with all components containerized and running on either Cloud Run or a VM. The deployment URLs are as follows:

- **ADK Classification API/UI:** https://receipt-classifier-32377413295.us-central1.run.app/
- **Flask API (Dashboard backend):** https://spendify-api-32377413295.us-central1.run.app/
- **Discord Bot:** Invite link - https://discord.com/oauth2/authorize?client_id=1375145987106144297&permissions=8&integration_type=0&scope=bot

Firebase is used for persistent storage, and the APIs use secure cloud endpoints. Deployment challenges included a steep learning curve for the GCP ADK pipeline due to limited documentation, as well as issues with integration, permissions, and service account setup. The team addressed these challenges, including cloud authentication, API versioning, and data hand-off edge cases.

Remaining steps include the completion of the dashboard frontend, integration of a regression model for spend prediction, and testing/patching for real-world OCR edge cases.

A screenshot of the partially completed dashboard is provided below.



3. Testing & Validation Evidence

Various testing methods have been employed:

- **Manual end-to-end runs:** The entire cycle from Discord upload to output and dashboard query has been tested.
- **Unit tests:** Core API, classification, and storage logic have been tested.
- **Model validation:** Holdout receipts were used to check the classification accuracy of the models.
- **User acceptance testing (UAT):** Real users have tested the Discord bot and provided feedback.

The team found that the Ollama-based LLM models were not accurate enough and replaced them with a GCP ADK agent pipeline. The classification results are robust for most receipt formats, API endpoints are stable, and data storage in Firebase is reliable.

Bugs that have been fixed include issues with JSON serialization for Firestore, improved error handling for bad receipts and cloud service errors, and a more consistent API response structure.

4. Final Plan & Documentation Readiness

The repository includes a detailed README, deployment guides, system diagrams, and code comments, ensuring the project is well-documented and easy to maintain. The final report and presentation slides are being drafted to clearly communicate the project's objectives, technical solutions, results, and lessons learned.

Future Plan's:

- **Polish the dashboard frontend** for usability and clear insights.
- **Integrate the regression model** into the pipeline and dashboard to enable spend forecasting.
- **Finalize and submit the report and presentation** per project guidelines.
- **Find bug then fixes with extensive testing** for a smooth, production-ready system.
- **Fix the AI chat Method** and Integrate through UI and Discord Bot.
- **Integrate New Features** and make it a Viable Product.

Remaining risks:
Potential cloud cost overruns, handling diverse OCR/data edge cases, and the need for rapid dashboard updates based on feedback.

Repository: https://github.com/jcp-tech/Spendify

Folder Structure:

```
Spendify/
├── discord_bot/              # Discord bot module
│   ├── bot.py                # Main Discord bot
│   ├── requirements.txt      # Bot dependencies
│   ├── deploy-bot.md         # Bot deployment guide
│   ├── img_content/          # Bot image content
│   └── README.md             # Bot documentation
├── flask_api/                # Main API server module
│   ├── main_api.py           # Flask API server
│   ├── gcp_docai.py          # OCR processing
│   ├── firebase_store.py     # Data storage
│   ├── gcp_adk_classification.py # ADK client
│   ├── requirements.txt      # API dependencies
│   ├── deploy-api.md         # API deployment guide
│   ├── Dockerfile            # Docker configuration
│   ├── dashboard/            # Web dashboard
│   │   ├── index.html        # Main dashboard
│   │   ├── login.html        # Login page
│   │   ├── register.html     # Registration page
│   │   ├── upload.html       # Upload page
│   │   ├── chat.html         # Chat interface
│   │   └── already_registered.html # Registration status
│   └── README.md             # API documentation
├── adk_pipeline/             # Agent Development Kit pipeline
│   ├── receipt_classifier/   # Agent pipeline
│   │   ├── agent.py          # Root agent orchestrator
│   │   ├── requirements.txt  # Agent dependencies
│   │   └── subagents/        # Individual processing agents
│   │       ├── classifier_init/      # Initial classification agent
│   │       │   └── agent.py          # Categorizes line items
│   │       ├── classification_grouper/ # Item grouping agent
│   │       │   ├── agent.py          # Groups items by category
│   │       │   └── tools.py          # Grouping utilities
│   │       ├── classification_reviewer/ # Validation agent
│   │       │   ├── agent.py          # Validates classifications
│   │       │   └── tools.py          # Review utilities
│   │       ├── classification_refiner/ # Correction agent
│   │       │   └── agent.py          # Refines misclassifications
│   │       └── classification_response/ # Final response agent
│   │           ├── agent.py          # Generates final summary
│   │           ├── tools.py          # Response utilities
│   │           └── firebase_store.py # Data persistence
│   ├── evals/                # Evaluation data
│   ├── deploy-adk.md         # ADK deployment guide
│   ├── deploy.sh             # Deployment script
│   ├── flow.png              # Pipeline flow diagram
│   └── README.md             # ADK documentation
├── process.png               # System process diagram
└── README.md                 # Project documentation
```